Exercise 2.1: My 2D Point Implementations

MyPoint.java:

```
public class MyPoint {
    private double x;
    private double y;

    public MyPoint() {
        this(0.0, 0.0);
    }

    public MyPoint(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public double distance(MyPoint other) {
        double dx = this.x - other.x;
        double dy = this.y - other.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    public static double distance(MyPoint p1, MyPoint p2) {
        double dx = p1.x - p2.x;
        double dy = p1.y - p2.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

TestMyPoint.java:

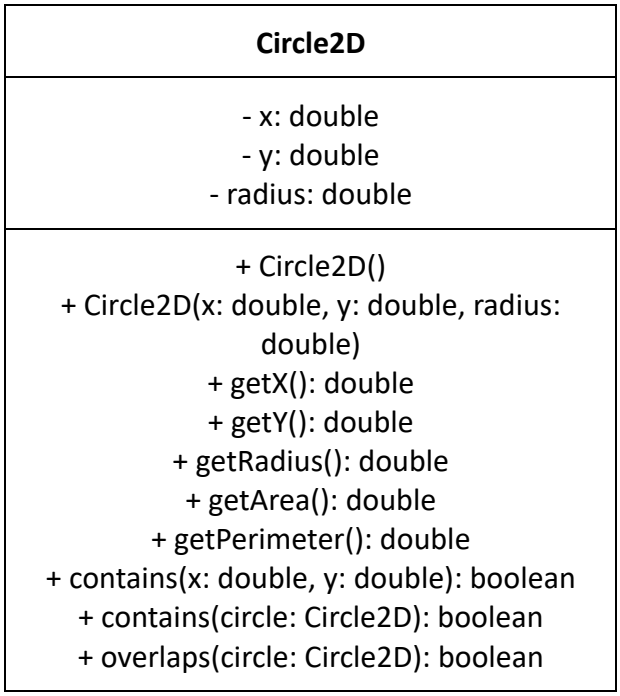
```
public class TestMyPoint {
    public static void main(String[] args) {
        MyPoint p1 = new MyPoint();
        MyPoint p2 = new MyPoint(10.25, 20.8);
        MyPoint p3 = new MyPoint(13.25, 24.8);

        System.out.println("Using instance method:");
        System.out.println("Distance from p1 to p2 = " + p1.distance(p2));
        System.out.println("Distance from p2 to p3 = " + p2.distance(p3));

        System.out.println("\nUsing static method:");
        System.out.println("Distance from p1 to p2 = " +
            MyPoint.distance(p1, p2));
        System.out.println("Distance from p2 to p3 = " +
            MyPoint.distance(p2, p3));
    }
}
```

Exercise 2.2: Circle2D Class

UML:



Circle2D.java:

```
public class Circle2D {
    private double x;
    private double y;
    private double radius;

    public Circle2D() {
        this(0, 0, 1);
    }

    public Circle2D(double x, double y, double radius) {
        this.x = x;
        this.y = y;
        this.radius = radius;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public double getRadius() {
        return radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
```

```
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    public boolean contains(double x, double y) {
        double dx = x - this.x;
        double dy = y - this.y;
        double distance = Math.sqrt(dx * dx + dy * dy);
        return distance <= radius;
    }

    public boolean contains(Circle2D circle) {
        double dx = circle.x - this.x;
        double dy = circle.y - this.y;
        double centerDistance = Math.sqrt(dx * dx + dy * dy);
        return centerDistance + circle.radius <= this.radius;
    }

    public boolean overlaps(Circle2D circle) {
        double dx = circle.x - this.x;
        double dy = circle.y - this.y;
        double centerDistance = Math.sqrt(dx * dx + dy * dy);
        return centerDistance < this.radius + circle.radius;
    }
}
```

TestCircle2D.java:

```
public class TestCircle2D {
    public static void main(String[] args) {
        Circle2D c1 = new Circle2D(2, 2, 5.5);

        System.out.println("Area = " + c1.getArea());
        System.out.println("Perimeter = " + c1.getPerimeter());

        System.out.println("c1.contains(3, 3) = " + c1.contains(3, 3));
        System.out.println("c1.contains(new Circle2D(4, 5, 10.5)) = "
                + c1.contains(new Circle2D(4, 5, 10.5)));
        System.out.println("c1.overlaps(new Circle2D(3, 5, 2.3)) = "
                + c1.overlaps(new Circle2D(3, 5, 2.3)));
    }
}
```

Exercise 2.3: Big Integers Divisible by 2 or 3

```
import java.math.BigInteger;

public class BigIntegerDivisible {
    public static void printFirst10FiftyDigitNumbersDivisibleBy2Or3() {
        // smallest 50-digit positive integer
        BigInteger number = BigInteger.TEN.pow(49);

        BigInteger two = BigInteger.valueOf(2);
        BigInteger three = BigInteger.valueOf(3);

        int count = 0;
        while (count < 10) {
            boolean divisibleBy2 = number.mod(two).equals(BigInteger.ZERO);
            boolean divisibleBy3 =
              number.mod(three).equals(BigInteger.ZERO);

            if (divisibleBy2 || divisibleBy3) {
                System.out.println(number);
                count++;
            }

            number = number.add(BigInteger.ONE);
        }
    }

    public static void main(String[] args) {
        printFirst10FiftyDigitNumbersDivisibleBy2Or3();
    }
}
```

Exercise 2.4: Triangle Class

Triangle.java:

```
public class Triangle extends GeometricObject {
    private double side1 = 1.0;
    private double side2 = 1.0;
    private double side3 = 1.0;

    public Triangle() {
        super();
    }

    public Triangle(double side1, double side2, double side3) {
        super();
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    public Triangle(double side1, double side2, double side3,
                    String color, boolean filled) {
        super(color, filled);
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    public double getSide1() {
        return side1;
    }

    public double getSide2() {
        return side2;
    }

    public double getSide3() {
        return side3;
    }

    public double getArea() {
        double s = getPerimeter() / 2.0;
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    public double getPerimeter() {
        return side1 + side2 + side3;
    }

    @Override
    public String toString() {
        return "Triangle: side1 = " + side1 + ", side2 = " + side2 +
          ", side3 = " + side3;
    }
}
```

TestTriangle.java:

```
import java.util.Scanner;

public class TestTriangle {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter side1: ");
        double side1 = input.nextDouble();

        System.out.print("Enter side2: ");
        double side2 = input.nextDouble();

        System.out.print("Enter side3: ");
        double side3 = input.nextDouble();

        input.nextLine(); // consume newline

        System.out.print("Enter color: ");
        String color = input.nextLine();

        System.out.print("Is filled (true/false): ");
        boolean filled = input.nextBoolean();

        Triangle triangle =
          new Triangle(side1, side2, side3, color, filled);

        System.out.println();
        System.out.println(triangle);
        System.out.println("Area = " + triangle.getArea());
        System.out.println("Perimeter = " + triangle.getPerimeter());
        System.out.println("Color = " + triangle.getColor());
        System.out.println("Filled = " + triangle.isFilled());
        System.out.println("Created on = " + triangle.getDateCreated());

        input.close();
    }
}
```

Exercise 2.5: Person, Student, Employee, Faculty, Staff

Person.java:

```
public class Person {
    private String name;
    private String address;
    private String phoneNumber;
    private String emailAddress;

    public Person(String name) {
        this(name, "", "", "");
    }

    public Person(String name,
                  String address,
                  String phoneNumber,
                  String emailAddress) {
        this.name = name;
        this.address = address;
        this.phoneNumber = phoneNumber;
        this.emailAddress = emailAddress;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }

    public String getPhoneNumber() {
        return phoneNumber;
    }

    public String getEmailAddress() {
        return emailAddress;
    }

    @Override
    public String toString() {
        return "Person: " + name;
    }
}
```

Student.java:

```
public class Student extends Person {
    public static final String FRESHMAN = "freshman";
    public static final String SOPHOMORE = "sophomore";
    public static final String JUNIOR = "junior";
    public static final String SENIOR = "senior";

    private String status;

    public Student(String name) {
        this(name, FRESHMAN);
    }

    public Student(String name, String status) {
        super(name);
        this.status = status;
    }

    public String getStatus() {
        return status;
    }

    @Override
    public String toString() {
        return "Student: " + getName();
    }
}
```

Employee.java:

```
import java.time.LocalDate;

public class Employee extends Person {
    private String office;
    private double salary;
    private LocalDate dateHired;

    public Employee(String name) {
        this(name, "", 0.0, LocalDate.now());
    }

    public Employee(String name,
                    String office,
                    double salary,
                    LocalDate dateHired) {
        super(name);
        this.office = office;
        this.salary = salary;
        this.dateHired = dateHired;
    }

    public String getOffice() {
        return office;
    }

    public double getSalary() {
        return salary;
    }

    public LocalDate getDateHired() {
        return dateHired;
    }

    @Override
    public String toString() {
        return "Employee: " + getName();
    }
}
```

Faculty.java:

```
import java.time.LocalDate;

public class Faculty extends Employee {
    private String officeHours;
    private String rank;

    public Faculty(String name) {
        this(name, "", 0.0, LocalDate.now(), "", "");
    }

    public Faculty(String name,
                   String office,
                   double salary,
                   LocalDate dateHired,
                   String officeHours,
                   String rank) {
        super(name, office, salary, dateHired);
        this.officeHours = officeHours;
        this.rank = rank;
    }

    public String getOfficeHours() {
        return officeHours;
    }

    public String getRank() {
        return rank;
    }

    @Override
    public String toString() {
        return "Faculty: " + getName();
    }
}
```

Staff.java:

```
import java.time.LocalDate;

public class Staff extends Employee {
    private String title;

    public Staff(String name) {
        this(name, "", 0.0, LocalDate.now(), "");
    }

    public Staff(String name,
                 String office,
                 double salary,
                 LocalDate dateHired,
                 String title) {
        super(name, office, salary, dateHired);
        this.title = title;
    }

    public String getTitle() {
        return title;
    }

    @Override
    public String toString() {
        return "Staff: " + getName();
    }
}
```

TestPeople.java:

```
import java.time.LocalDate;

public class TestPeople {
    public static void main(String[] args) {
        Person[] people = new Person[5];

        people[0] = new Person("Alice");
        people[1] = new Student("Bob", Student.JUNIOR);
        people[2] = new Employee("Carol", "Room 201", 5000,
                LocalDate.of(2023, 9, 1));
        people[3] = new Faculty("David", "Room 305", 8000,
                LocalDate.of(2021, 2, 15), "Mon 2-4pm", "Professor");
        people[4] = new Staff("Eva", "Office 101", 4500,
                LocalDate.of(2022, 6, 1), "Administrator");

        for (Person p : people) {
            System.out.println(p.toString());
        }
    }
}
```

Answer to the access modifier question:

`name` should be `private`.

Exercise 2.6: MyStack using Inheritance

MyStack.java:

```
import java.util.ArrayList;

public class MyStack extends ArrayList<Object> {
    public boolean isEmpty() {
        return super.isEmpty();
    }

    public int getSize() {
        return size();
    }

    public Object peek() {
        if (isEmpty()) {
            return null;
        }
        return get(getSize() - 1);
    }

    public Object pop() {
        if (isEmpty()) {
            return null;
        }
        return remove(getSize() - 1);
    }

    public void push(Object o) {
        add(o);
    }

    public int search(Object o) {
        int index = lastIndexOf(o);
        if (index == -1) {
            return -1;
        }
        return getSize() - index;
    }

    @Override
    public String toString() {
        return "stack: " + super.toString();
    }
}
```

TestMyStack.java:

```
import java.util.Scanner;

public class TestMyStack {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        MyStack stack = new MyStack();

        System.out.println("Enter five strings:");
        for (int i = 0; i < 5; i++) {
            stack.push(input.nextLine());
        }

        System.out.println("Strings in reverse order:");
        while (!stack.isEmpty()) {
            System.out.println(stack.pop());
        }

        input.close();
    }
}
```

Exercise 2.7: Tax Class

UML:

Tax
+ SINGLE_FILER: int + MARRIED_JOINTLY_OR_QUALIFYING_WIDOW_ER: int + MARRIED_SEPARATELY: int + HEAD_OF_HOUSEHOLD: int - filingStatus: int - brackets: int[][] - rates: double[] - taxableIncome: double
+ Tax() + Tax(filingStatus: int, brackets: int[][], rates: double[], taxableIncome: double) + getFilingStatus(): int + setFilingStatus(filingStatus: int): void + getBrackets(): int[][] + setBrackets(brackets: int[][]): void + getRates(): double[] + setRates(rates: double[]): void + getTaxableIncome(): double + setTaxableIncome(taxableIncome: double): void + getTax(): double

Tax.java:

```
public class Tax {
    public static final int SINGLE_FILER = 0;
    public static final int MARRIED_JOINTLY_OR_QUALIFYING_WIDOW_ER = 1;
    public static final int MARRIED_SEPARATELY = 2;
    public static final int HEAD_OF_HOUSEHOLD = 3;

    private int filingStatus;
    private int[][] brackets;
    private double[] rates;
    private double taxableIncome;

    public Tax() {
    }

    public Tax(int filingStatus, int[][] brackets,
      double[] rates, double taxableIncome) {
        this.filingStatus = filingStatus;
        this.brackets = brackets;
        this.rates = rates;
        this.taxableIncome = taxableIncome;
    }
```

```
    public int getFilingStatus() {
        return filingStatus;
    }

    public void setFilingStatus(int filingStatus) {
        this.filingStatus = filingStatus;
    }

    public int[][] getBrackets() {
        return brackets;
    }

    public void setBrackets(int[][] brackets) {
        this.brackets = brackets;
    }

    public double[] getRates() {
        return rates;
    }

    public void setRates(double[] rates) {
        this.rates = rates;
    }

    public double getTaxableIncome() {
        return taxableIncome;
    }

    public void setTaxableIncome(double taxableIncome) {
        this.taxableIncome = taxableIncome;
    }

    public double getTax() {
        double tax = 0;
        int[] bracket = brackets[filingStatus];

        for (int i = 0; i < rates.length - 1; i++) {
            if (taxableIncome > bracket[i]) {
                double upperBound =
                  Math.min(taxableIncome, bracket[i + 1]);
                tax += (upperBound - bracket[i]) * rates[i];
            } else {
                return tax;
            }
        }

        if (taxableIncome > bracket[rates.length - 1]) {
            tax += (taxableIncome - bracket[rates.length - 1]) *
              rates[rates.length - 1];
        }

        return tax;
    }
}
```

TestTax.java:

```
public class TestTax {
    public static void main(String[] args) {
```

```
        int[][] brackets2001 = {
         {0, 27050, 65550, 136750, 297350},   // single
         {0, 45200, 109250, 166500, 297350},  // married jointly
         {0, 22600, 54625, 83250, 148675},    // married separately
         {0, 36250, 93650, 151650, 297350}    // head of household
        };

        double[] rates2001 = {0.15, 0.275, 0.305, 0.355, 0.391};

        int[][] brackets2009 = {
         {0, 8350, 33950, 82250, 171550, 372950},     // single
         {0, 16700, 67900, 137050, 208850, 372950},   // married jointly
         {0, 8350, 33950, 68525, 104425, 186475},     // married separately
         {0, 11950, 45500, 117450, 190200, 372950}    // head of household
        };

        double[] rates2009 = {0.10, 0.15, 0.25, 0.28, 0.33, 0.35};

        System.out.println("2001 tax tables");
        printTaxTable(brackets2001, rates2001);

        System.out.println();
        System.out.println("2009 tax tables");
        printTaxTable(brackets2009, rates2009);
    }

    public static void printTaxTable(int[][] brackets, double[] rates) {
        System.out.printf("%-10s%-15s%-15s%-15s%-15s%n",
          "Income", "Single", "MarriedJoint", "MarriedSep", "HeadHouse");

        for (int income = 50000; income <= 60000; income += 1000) {
            System.out.printf("%-10d", income);

            for (int status = 0; status < 4; status++) {
                Tax tax = new Tax(status, brackets, rates, income);
                System.out.printf("%-15.2f", tax.getTax());
            }

            System.out.println();
        }
    }
}
```